

30538 Problem Set 4: Web Scraping

Lisa Liang

2026-02-07

Due 02/07 at 5:00PM Central.

“This submission is my work alone and complies with the 30538 integrity policy.” Add your initials to indicate your agreement: YL

Github Classroom Assignment Setup and Submission Instructions

1. Accepting and Setting up the PS4 Assignment Repository

- Each student must individually accept the repository for the problem set from Github Classroom (“ps4”) – <https://classroom.github.com/a/hWhcHqH>
 - You will be prompted to select your cnetid from the list in order to link your Github account to your cnetid.
 - If you can’t find your cnetid in the link above, click “continue to next step” and accept the assignment, then add your name, cnetid, and Github account to this Google Sheet and we will manually link it: <https://rb.gy/9u7fb6>
- If you authenticated and linked your Github account to your device, you should be able to clone your PS4 assignment repository locally.
- Contents of PS4 assignment repository:
 - `ps4_template.qmd`: this is the Quarto file with the template for the problem set. You will write your answers to the problem set here.

2. Submission Process:

- Knit your completed solution `ps4.qmd` as a pdf `ps4.pdf`.
 - Your submission does not need runnable code. Instead, you will tell us either what code you ran or what output you got.
- To submit, push `ps4.qmd` and `ps4.pdf` to your PS4 assignment repository. Confirm on Github.com that your work was successfully pushed.

Grading

- You will be graded on what was last pushed to your PS4 assignment repository before the assignment deadline
- Problem sets will be graded for completion as: {missing (0%); - (incomplete, 50%); + (excellent, 100%)}
 - The percent values assigned to each problem denote how long we estimate the problem will take as a share of total time spent on the problem set, not the points they are associated with.
- In order for your submission to be considered complete, you need to push both your `ps4.qmd` and `ps4.pdf` to your repository. Submissions that do not include both files will automatically receive 50% credit.

```

import pandas as pd
import altair as alt
import time

import warnings
warnings.filterwarnings('ignore')
alt.renderers.enable("png")

```

```

RendererRegistry.enable('png')

```

Step 1: Develop initial scraper and crawler

```

import requests
from bs4 import BeautifulSoup
import pandas as pd
from urllib.parse import urljoin

URL = "https://oig.hhs.gov/fraud/enforcement/"

resp = requests.get(URL, timeout=30)
resp.raise_for_status()

soup = BeautifulSoup(resp.text, "html.parser")

rows = []

category_selectors = ["h2", "h3"]
current_category = None
root = soup.find("main") or soup

for tag in root.find_all(category_selectors + ["a"], recursive=True):
    if tag.name in category_selectors:
        text = tag.get_text(strip=True)
        if text:
            current_category = text
            continue

    if tag.name == "a":
        title = tag.get_text(" ", strip=True)
        href = tag.get("href")

```

```

        if not title or not href:
            continue

        full_link = urljoin(URL, href)

        if "/fraud/enforcement/" not in full_link:
            continue
        if full_link.rstrip("/") == URL.rstrip("/"):
            continue

        container = tag.find_parent(["li", "div", "article", "section"]) or
↪ tag.parent

        date = None
        time_tag = container.find("time")
        if time_tag and time_tag.get_text(strip=True):
            date = time_tag.get_text(strip=True)
        else:

            candidates = container.find_all(["span", "p", "div"], limit=6)
            for c in candidates:
                t = c.get_text(" ", strip=True)
                if any(m in t for m in ["Jan", "Feb", "Mar", "Apr", "May",
↪ "Jun",
                                         "Jul", "Aug", "Sep", "Oct", "Nov",
                                         ↪ "Dec"]) or \
                    any(ch.isdigit() for ch in t) and len(t) <= 30:
                    date = t
                    break

        rows.append({
            "title": title,
            "date": date,
            "category": current_category,
            "link": full_link
        })

df = pd.DataFrame(rows)

df = (
    df.dropna(subset=["title", "link"])
    .drop_duplicates(subset=["title", "link"])
    .reset_index(drop=True)

```

```
)
```

```
print(df.head())
```

```
                                title \
0                                About Enforcement Actions
1                                Civil Monetary Penalty Authorities
2  Houston Transplant Doctor Indicted For Making ...
3  MultiCare Health System to Pay Millions to Set...
4  Brooklyn Banker Pleads Guilty to Laundering Pr...

                                date \
0                                None
1                                None
2  February 5, 2026 Criminal and Civil Actions
3  February 4, 2026 Criminal and Civil Actions
4  February 3, 2026 COVID-19

                                category \
0                                In This Section
1                                In This Section
2  Houston Transplant Doctor Indicted For Making ...
3  MultiCare Health System to Pay Millions to Set...
4  Brooklyn Banker Pleads Guilty to Laundering Pr...

                                link
0  https://oig.hhs.gov/fraud/enforcement/about/
1  https://oig.hhs.gov/fraud/enforcement/civil-mo...
2  https://oig.hhs.gov/fraud/enforcement/houston-...
3  https://oig.hhs.gov/fraud/enforcement/multicar...
4  https://oig.hhs.gov/fraud/enforcement/brooklyn...
```

Step 2: Making the scraper dynamic

1. Turning the scraper into a function

- a. Pseudo-Code

Inputs: start_year, start_month, run_scraper (bool)

Guardrail: If start_year < 2013: print “restrict to >= 2013” and return None.

Set start date cutoff: start_date = datetime(start_year, start_month, 1)

Initialize: page = 1, rows = [], keep_going = True

while loop:

- * Build URL:

 - if page == 1: base URL

 - else: base URL + ?page=page

- * Request page HTML

- * Parse all enforcement "items" on the page

- * If no items found → break (end of pagination)

- * For each item:

 - Extract title, date, link

 - Track the current category heading (e.g., most recent < h2 >/< h3 >)

 - Convert date string → datetime

 - If date < start_date: set keep_going = False

 - Else: append row to rows

- * page += 1

- * sleep(1) to be polite

- * if keep_going == False: break

Make tidy dataframe with columns: title, date, category, link

Sort by date (optional), save to CSV: enforcement_actions_< year >_< month >.csv

Return dataframe

We are using a while loop because we don't know how many pages you need ahead of time. It will stop when it hits an old enough date or there's no next page.

- b. Create Dynamic Scraper

```

import time
import re
from datetime import datetime
from urllib.parse import urljoin

import requests
import pandas as pd
from bs4 import BeautifulSoup
from dateutil import parser as dateparser # pip install python-dateutil
↳ (often already installed)

BASE_URL = "https://oig.hhs.gov/fraud/enforcement/"

def scrape_enforcement_actions_since(start_year: int,
                                     start_month: int,
                                     run_scraper: bool = False) ->
↳ pd.DataFrame | None:
    """
    Scrape HHS OIG enforcement actions from (start_year, start_month) through
    ↳ today.
    Saves to: enforcement_actions_<year>_<month>.csv
    If run_scraper is False, does nothing (so knitting is fast).
    """

    if not run_scraper:
        print("run_scraper=False, skipping scrape (use saved CSV instead).")
        return None

    if start_year < 2013:
        print("Year must be >= 2013 (HHS OIG enforcement actions only listed
        ↳ after 2013).")
        return None

    if start_month < 1 or start_month > 12:
        raise ValueError("start_month must be between 1 and 12.")

    start_date = datetime(start_year, start_month, 1)

    session = requests.Session()
    session.headers.update({
        "User-Agent": "Mozilla/5.0 (compatible; course scraper;
        ↳ +https://oig.hhs.gov/)"

```

```

})

rows = []
page = 1
keep_going = True

while keep_going:
    url = BASE_URL if page == 1 else f"{BASE_URL}?page={page}"
    r = session.get(url, timeout=30)
    r.raise_for_status()

    soup = BeautifulSoup(r.text, "html.parser")
    main = soup.find("main") or soup

    # Strategy:
    # - Walk through headings (h2/h3) to capture category
    # - Collect links to individual enforcement action pages
    current_category = None
    items_found_on_page = 0

    # We scan in document order to "carry" the category heading down
    for node in main.find_all(["h2", "h3", "a"], recursive=True):

        if node.name in ["h2", "h3"]:
            txt = node.get_text(" ", strip=True)
            if txt:
                current_category = txt
                continue

        if node.name == "a":
            title = node.get_text(" ", strip=True)
            href = node.get("href")
            if not title or not href:
                continue

            full_link = urljoin(BASE_URL, href)

            # Keep only links that look like individual enforcement
            ↪ entries
            # (This filter is important to avoid grabbing nav/footer
            ↪ links.)
            if "/fraud/enforcement/" not in full_link:
                continue

```



```

        if full_link.rstrip("/") == BASE_URL.rstrip("/"):
            continue

        container = node.find_parent(["li", "article", "div",
↪ "section"]) or node.parent

        # Try to extract date: prefer <time>, fallback to nearby text
        date_text = None
        time_tag = container.find("time")
        if time_tag:
            date_text = time_tag.get("datetime") or
↪ time_tag.get_text(" ", strip=True)

        if not date_text:
            # fallback: search nearby text for a date-like string
            nearby = container.get_text(" ", strip=True)
            # crude pattern, but works as a fallback:
            # e.g., "January 12, 2024"
            m =
↪ re.search(r"(Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec) [a-z]*\s+\d{1,2},\s+\d{4}",
↪ nearby)

            if m:
                date_text = m.group(0)

        if not date_text:
            # If date is missing, skip (or keep with None if you
↪ prefer)
            continue

        try:
            dt = dateparser.parse(date_text).replace(tzinfo=None)
        except Exception:
            continue

        items_found_on_page += 1

        # Stop condition: once we hit older than start_date, we can
↪ stop after this page scan
        if dt < start_date:
            keep_going = False
            continue

```

```

        rows.append({
            "title": title,
            "date": dt.date().isoformat(),    # tidy ISO date
            "category": current_category,
            "link": full_link
        })

    if items_found_on_page == 0:
        # no enforcement items recognized => likely end of pagination or
        # selector mismatch
        break

    page += 1
    time.sleep(1)

    df = pd.DataFrame(rows).drop_duplicates(subset=["title",
↪ "link"]).reset_index(drop=True)

    # Sort so "earliest scraped" is easy to find
    if not df.empty:
        df = df.sort_values("date").reset_index(drop=True)

    outname = f"enforcement_actions_{start_year}_{start_month:02d}.csv"
    df.to_csv(outname, index=False)
    print(f"Saved {len(df)} rows to {outname}")

    return df

```

```

RUN_SCRAPER = True
df_2024 = scrape_enforcement_actions_since(2024, 1, run_scraper=RUN_SCRAPER)

len_2024 = len(df_2024)
earliest_2024 = df_2024.iloc[0]

len_2024, earliest_2024.to_dict()

```

Saved 1787 rows to enforcement_actions_2024_01.csv

```

(1787,
 {'title': 'Former Nurse Aide Indicted In Death Of Clarksville Patient
Arrested In Georgia',
  'date': '2024-01-03',

```

```
'category': 'Former Nurse Aide Indicted In Death Of Clarksville Patient  
Arrested In Georgia',  
'link':  
'https://oig.hhs.gov/fraud/enforcement/former-nurse-aide-indicted-in-death-of-clarksville-patient-arrested-in-georgia'
```

```
RUN_SCRAPER = False # turn True only when we actually want to scrape  
  
df_2024 = scrape_enforcement_actions_since(2024, 1, run_scraper=RUN_SCRAPER)
```

run_scraper=False, skipping scrape (use saved CSV instead).

- c. Test Your Code

```
RUN_SCRAPER = True  
df_2022 = scrape_enforcement_actions_since(2022, 1, run_scraper=RUN_SCRAPER)  
  
len_2022 = len(df_2022)  
earliest_2022 = df_2022.iloc[0]  
  
len_2022, earliest_2022.to_dict()
```

Saved 3377 rows to enforcement_actions_2022_01.csv

```
(3377,  
{  
  'title': 'Integrated Pain Management Medical Group Agreed to Pay $10,000  
for Allegedly Violating the Civil Monetary Penalties Law by Employing  
Excluded Individuals',  
  'date': '2022-01-04',  
  'category': 'Integrated Pain Management Medical Group Agreed to Pay $10,000  
for Allegedly Violating the Civil Monetary Penalties Law by Employing  
Excluded Individuals',  
  'link':  
  'https://oig.hhs.gov/fraud/enforcement/integrated-pain-management-medical-group-agreed-to-pay-10000-for-allegedly-violating-the-civil-monetary-penalties-law-by-employing-excluded-individuals'
```

```
RUN_SCRAPER = False
```

Step 3: Plot data based on scraped data

1. Plot the number of enforcement actions over time

```
import pandas as pd
import altair as alt

df =
    ↪ pd.read_csv("/Users/Swaggy-LYX/Downloads/enforcement_actions_2022_01.csv")

df["date"] = pd.to_datetime(df["date"])

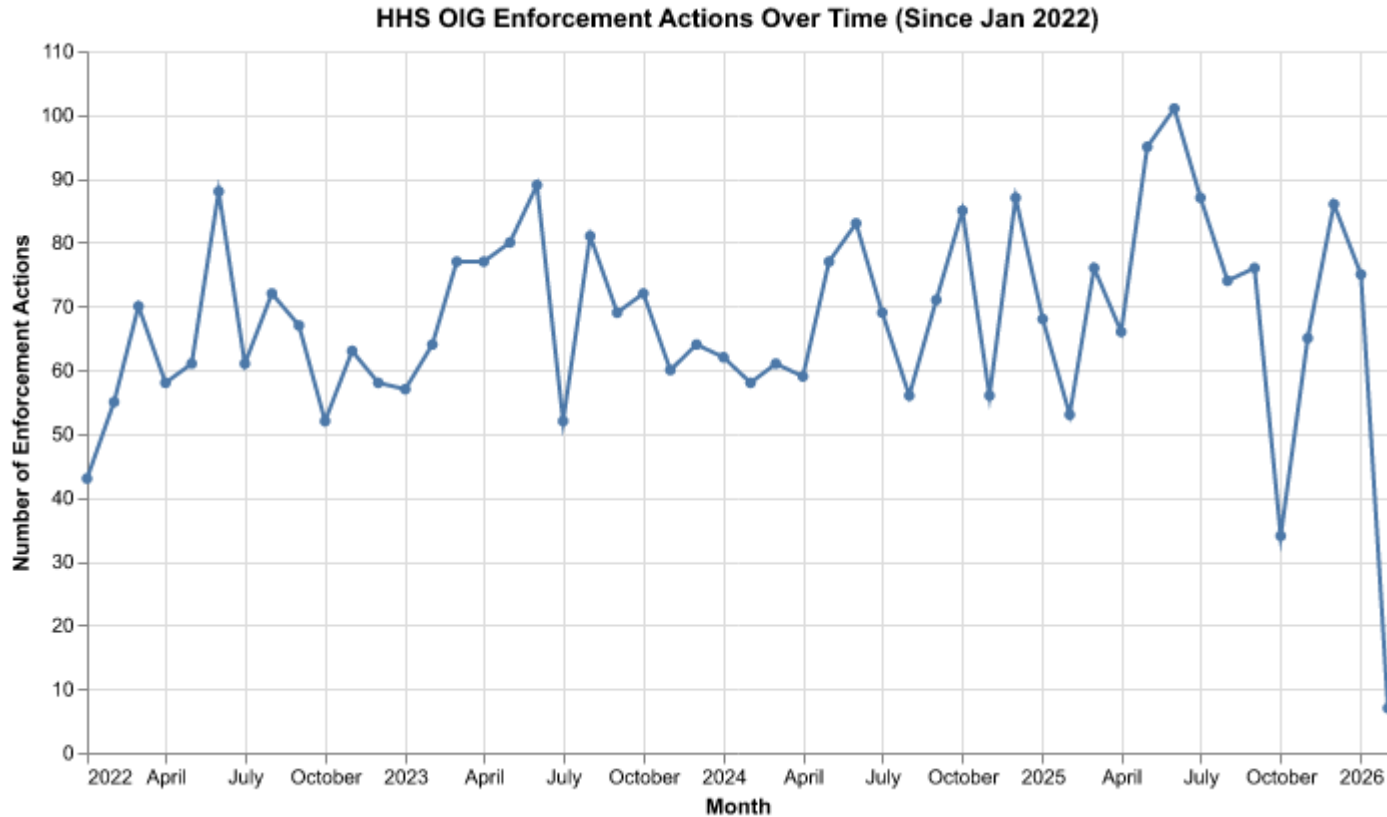
df.head()
```

	title	date	category
0	Integrated Pain Management Medical Group Agree...	2022-01-04	Integrated Pain Management Medical
1	Ohio home healthcare provider agrees to pay \$5...	2022-01-04	Ohio home healthcare provider agrees
2	Central Medical Systems, LLC, Alan Trent Harle...	2022-01-04	Central Medical Systems, LLC, Alan T
3	Emergency Ambulance Service Agreed to Pay \$430...	2022-01-05	Emergency Ambulance Service Agreed
4	Attorney General Alan Wilson announces arrest ...	2022-01-05	Attorney General Alan Wilson announ

```
df["year_month"] = df["date"].dt.to_period("M").dt.to_timestamp()

monthly_counts = (
    df
    .groupby("year_month")
    .size()
    .reset_index(name="n_actions")
)
```

```
alt.Chart(monthly_counts).mark_line(point=True).encode(
    x=alt.X("year_month:T", title="Month"),
    y=alt.Y("n_actions:Q", title="Number of Enforcement Actions"),
    tooltip=["year_month:T", "n_actions:Q"]
).properties(
    title="HHS OIG Enforcement Actions Over Time (Since Jan 2022)",
    width=650,
    height=350
)
```



2. Plot the number of enforcement actions categorized:

- based on “Criminal and Civil Actions” vs. “State Enforcement Agencies”

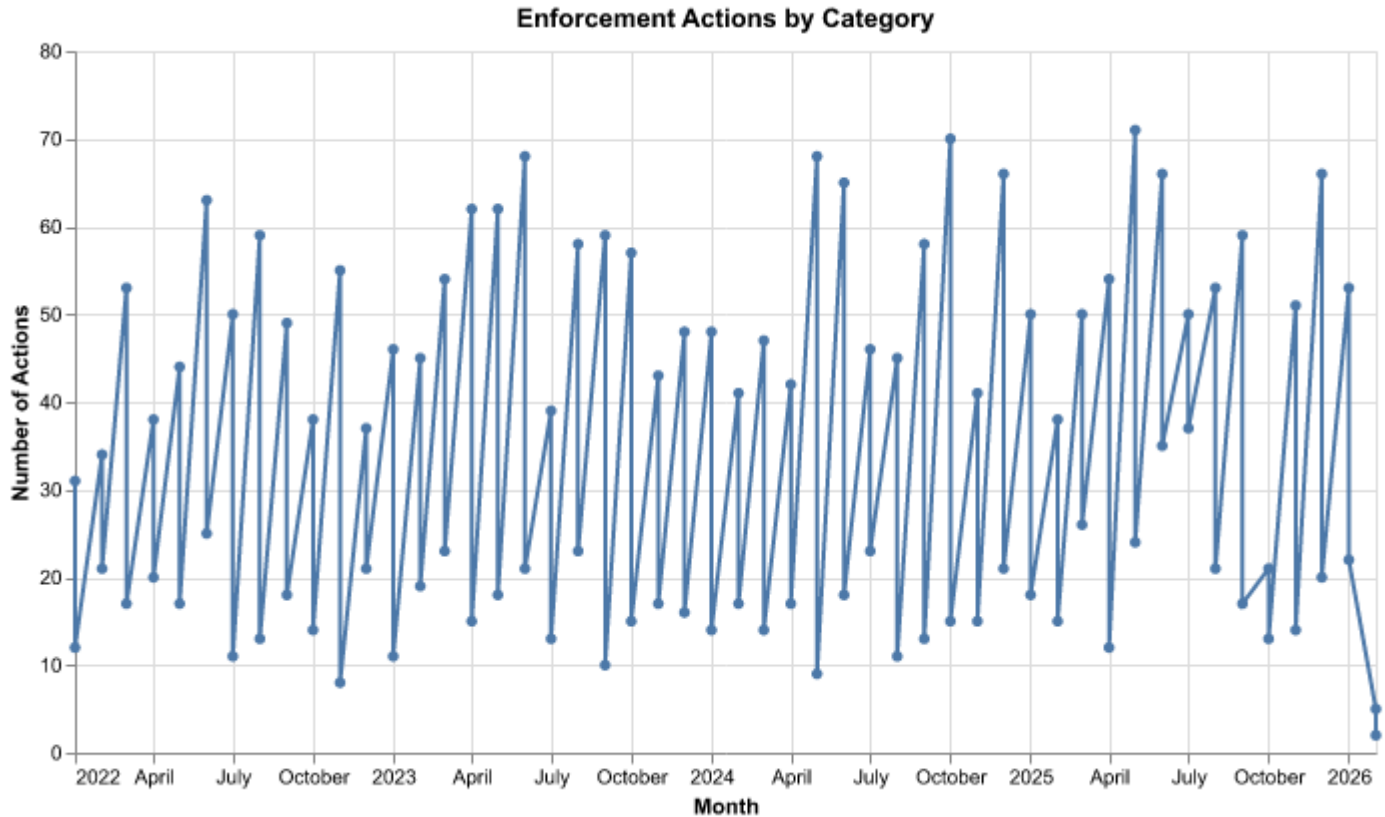
```
def classify_category_from_title(title: str) -> str:
    t = str(title).lower()
    state_markers = [
        "attorney general", "ag ", "medicaid fraud control unit", "mfcu",
        "state", "commonwealth", "department of justice", "doj",
        "office of inspector general", "state of", "kentucky", "ohio",
        ↪ "texas",
        "florida", "new york", "california" # optional: remove if too
        ↪ aggressive
    ]
    if any(k in t for k in state_markers):
        return "State Enforcement Agencies"
    return "Criminal and Civil Actions"
```

```
df["category_fixed"] = df["title"].apply(classify_category_from_title)
df["category_fixed"].value_counts()
```

```
category_fixed
Criminal and Civil Actions    2516
State Enforcement Agencies    861
Name: count, dtype: int64
```

```
monthly_by_category = (
    df.groupby(["year_month", "category_fixed"])
        .size()
        .reset_index(name="n_actions")
)
```

```
alt.Chart(monthly_by_category).mark_line(point=True).encode(
    x=alt.X("year_month:T", title="Month"),
    y=alt.Y("n_actions:Q", title="Number of Actions"),
    color=alt.Color("category:N", title="Category"),
    tooltip=["year_month:T", "category:N", "n_actions:Q"]
).properties(
    title="Enforcement Actions by Category",
    width=650,
    height=350
)
```



- based on five topics

```
cca = df[df["category_fixed"] == "Criminal and Civil Actions"].copy()
```

```
def classify_topic(title):
    title = title.lower()

    if any(word in title for word in ["health", "medicare", "medicaid",
        ↪ "hospital", "physician", "clinic"]):
        return "Health Care Fraud"
    elif any(word in title for word in ["bank", "financial", "loan", "wire",
        ↪ "securities", "money laundering"]):
        return "Financial Fraud"
    elif any(word in title for word in ["drug", "opioid", "pharmacy",
        ↪ "controlled substance"]):
        return "Drug Enforcement"
    elif any(word in title for word in ["bribe", "corrupt", "kickback"]):
        return "Bribery/Corruption"
```

```

        else:
            return "Other"

cca["topic"] = cca["title"].apply(classify_topic)

cca["topic"].value_counts()

monthly_by_topic = (
    cca
    .groupby(["year_month", "topic"])
    .size()
    .reset_index(name="n_actions")
)

alt.Chart(monthly_by_topic).mark_line(point=True).encode(
    x=alt.X("year_month:T", title="Month"),
    y=alt.Y("n_actions:Q", title="Number of Actions"),
    color=alt.Color("topic:N", title="Topic"),
    tooltip=["year_month:T", "topic:N", "n_actions:Q"]
).properties(
    title="Criminal and Civil Enforcement Actions by Topic",
    width=650,
    height=350
)

```